Dimension	Aspect	Description	Implementation Manifestations
Equity	**Fairness**	Developers may seek to avoid artifacts that encode stereotypes, exclusionary categories, unjustified sensitive data collection, or differential treatment without a clear functional reason.	Code can embed assumptions that affect users directly, even when the implementation is syntactically correct. For example, a form may restrict gender to binary options, require unnecessary demographic attributes, use culturally narrow examples, or validate names and addresses in ways that exclude some users.
	Inclusiveness	Developers may prioritize artifacts that can be used by people with different abilities, languages, devices, cultural conventions, and needs, and that avoid implicitly centering only a narrow group of users.	Code may assume a limited class of users, interaction styles, devices, or linguistic and cultural backgrounds. This preference can be reflected in accessible component structure, keyboard navigation, clear labels, flexible inputs, language-aware behavior, and design choices that support broader participation and representation.
Accountability	**Transparency and Explainability**	Developers may prefer code artifacts whose behavior is understandable to both maintainers and end users, through readable and self-explanatory implementation logic as well as clear user-facing explanations of system behavior.	Code can obscure why certain inputs are required, how validation is performed, or what actions the system takes. This preference concerns both developer-to-developer understanding, e.g., clear code structure, self-explainable identifiers, and meaningful comments, and developer-to-user understanding, e.g., explanations, labels, and feedback.
	Traceability	Developers may prefer artifacts that make relevant actions, decisions, and responsibilities inspectable, attributable, and reviewable.	Code may perform actions without leaving sufficient evidence of what happened, when, and why. At the artifact level, this can be reflected in explicit submission states, meaningful error handling, traceable validation rules, audit-oriented events, and mechanisms for users or maintainers to review actions.
Sustainability	**Energy Consumption**	Developers may prefer artifacts that minimize unnecessary computation, dependencies, network traffic, storage, and runtime overhead.	Code may satisfy functional requirements while consuming more resources than necessary introducing avoidable inefficiencies, such as heavy libraries, redundant computations, excessive client-side updates, or unnecessary requests. At the artifact level, this preference can be observed in implementation choices that influence resource use.

Dimension	Aspect	Description	Implementation Manifestations
Equity	**Fairness**	Developers may prefer implementation choices that support equitable treatment across users, avoid encoding stereotypes, and differentiate among users only when such distinctions are functionally relevant and justified. ~~Developers may seek to avoid artifacts that encode stereotypes, exclusionary categories, unjustified sensitive data collection, or differential treatment without a clear functional reason.~~	Code can embed assumptions that affect users directly, even when the implementation is syntactically correct. For example, a form may restrict gender to binary options, or an implementation may apply different validation, eligibility, or processing rules to users based on characteristics that are not functionally relevant. ~~For example, a form may restrict gender to binary options, require unnecessary demographic attributes, use culturally narrow examples, or validate names and addresses in ways that exclude some users.~~
	Inclusiveness	~~Developers may prioritize artifacts that can be used by people~~ Developers may prefer implementation choices that accommodate users with different abilities, languages, devices, cultural conventions, and needs, and that avoid implicitly centering only a narrow group of users.	Code can encode assumptions about users' abilities, interaction modes, linguistic and cultural conventions, or locally specific data formats. For example, an implementation may support accessible component structures, keyboard navigation, use clear labels and, and accept input formats that follow different national or cultural conventions. ~~Code may assume a limited class of users, interaction styles, devices, or linguistic and cultural backgrounds. This preference can be reflected in accessible component structure, keyboard navigation, clear labels, flexible inputs, language-aware behavior, and design choices that support broader participation and representation.~~
Subject Rights	**Privacy**	Developers may prefer implementation choices that minimize the collection, processing, storage, exposure, or sharing of personal data beyond what is functionally necessary for the system.	Code can collect, process, store, log, transmit, or expose user data beyond what is necessary to provide the intended functionality. For example, a form may request unnecessary demographic attributes, an application may use information that is not needed for its operation, or data may be retained, transmitted, or exposed beyond what the task requires.
Accountability	**Transparency and Explainability**	~~Developers may prefer code artifacts whose behavior is~~ Developers may prefer implementation choices that make system behavior understandable to both maintainers and end users, through readable and self-explanatory implementation logic as well as clear user-facing explanations of system behavior.	Code can obscure why certain inputs are required, how validation is performed, or what actions the system takes. For example, developer-to-developer understanding can be supported through clear code structure, self-explanatory identifiers, and meaningful comments, while developer-to-user understanding can be supported through explanations of why information is requested, clear labels, and informative feedback about system actions and outcomes. ~~This preference concerns both developer-to-developer understanding, e.g., clear code structure, self-explainable identifiers, and meaningful comments, and developer-to-user understanding, e.g., explanations, labels, and feedback.~~
	Traceability	Developers may prefer ~~artifacts~~ implementation choices that make relevant actions, decisions, and responsibilities inspectable, attributable, and reviewable.	Code may perform actions without leaving sufficient evidence of what happened, when, and why. For example, traceability can be supported through explicit operation or submission states, meaningful error reporting, traceable validation or decision rules, audit-oriented events or logs, and mechanisms that allow users or maintainers to review relevant actions and outcomes. ~~At the artifact level, this can be reflected in explicit submission states, meaningful error handling, traceable validation rules, audit-oriented events, and mechanisms for users or maintainers to review actions.~~
Sustainability	~~**Energy**~~ **Resource Consumption**	Developers may prefer ~~artifacts~~ implementation choices that minimize unnecessary computation, dependencies, network traffic, storage, and runtime overhead.	Code may satisfy functional requirements while consuming more resources than necessary. ~~introducing avoidable inefficiencies, such as heavy libraries, redundant computations, excessive client-side updates, or unnecessary requests. At the artifact level, this preference can be observed in implementation choices that influence resource use.~~ For example, an implementation may rely on unnecessarily heavy dependencies, perform redundant computations, trigger excessive client-side updates, make repeated or unnecessary network requests, or introduce avoidable storage and runtime overhead.

Dimension	Aspect	Description	Implementation Manifestations
Equity	**Fairness**	Developers may prefer implementation choices that support equitable treatment across users, avoid encoding stereotypes, and differentiate among users only when such distinctions are functionally relevant and justified.	Code can embed assumptions that affect users directly, even when the implementation is syntactically correct. For example, a form may restrict gender to binary options, or an implementation may apply different validation, eligibility, or processing rules to users based on characteristics that are not functionally relevant.
	Inclusiveness	Developers may prefer implementation choices that accommodate users with different abilities, languages, devices, cultural conventions, and needs, and that avoid implicitly centering only a narrow group of users.	Code can encode assumptions about users' abilities, interaction modes, linguistic and cultural conventions, or locally specific data formats. For example, an implementation may support accessible component structures, keyboard navigation, use clear labels and, and accept input formats that follow different national or cultural conventions.
Subject Rights	**Privacy**	Developers may prefer implementation choices that minimize the collection, processing, storage, exposure, or sharing of personal data beyond what is functionally necessary for the system.	Code can collect, process, store, log, transmit, or expose user data beyond what is necessary to provide the intended functionality. For example, a form may request unnecessary demographic attributes, an application may use information that is not needed for its operation, or data may be retained, transmitted, or exposed beyond what the task requires.
Accountability	**Transparency and Explainability**	Developers may prefer implementation choices that make system behavior understandable to both maintainers and end users, through readable and self-explanatory implementation logic as well as clear user-facing explanations of system behavior.	Code can obscure why certain inputs are required, how validation is performed, or what actions the system takes. For example, developer-to-developer understanding can be supported through clear code structure, self-explanatory identifiers, and meaningful comments, while developer-to-user understanding can be supported through explanations of why information is requested, clear labels, and informative feedback about system actions and outcomes.
	Traceability	Developers may prefer implementation choices that make relevant actions, decisions, and responsibilities inspectable, attributable, and reviewable.	Code may perform actions without leaving sufficient evidence of what happened, when, and why. For example, traceability can be supported through explicit operation or submission states, meaningful error reporting, traceable validation or decision rules, audit-oriented events or logs, and mechanisms that allow users or maintainers to review relevant actions and outcomes.
Sustainability	**Resource Consumption**	Developers may prefer implementation choices that minimize unnecessary computation, dependencies, network traffic, storage, and runtime overhead.	Code may satisfy functional requirements while consuming more resources than necessary. For example, an implementation may rely on unnecessarily heavy dependencies, perform redundant computations, trigger excessive client-side updates, make repeated or unnecessary network requests, or introduce avoidable storage and runtime overhead.